



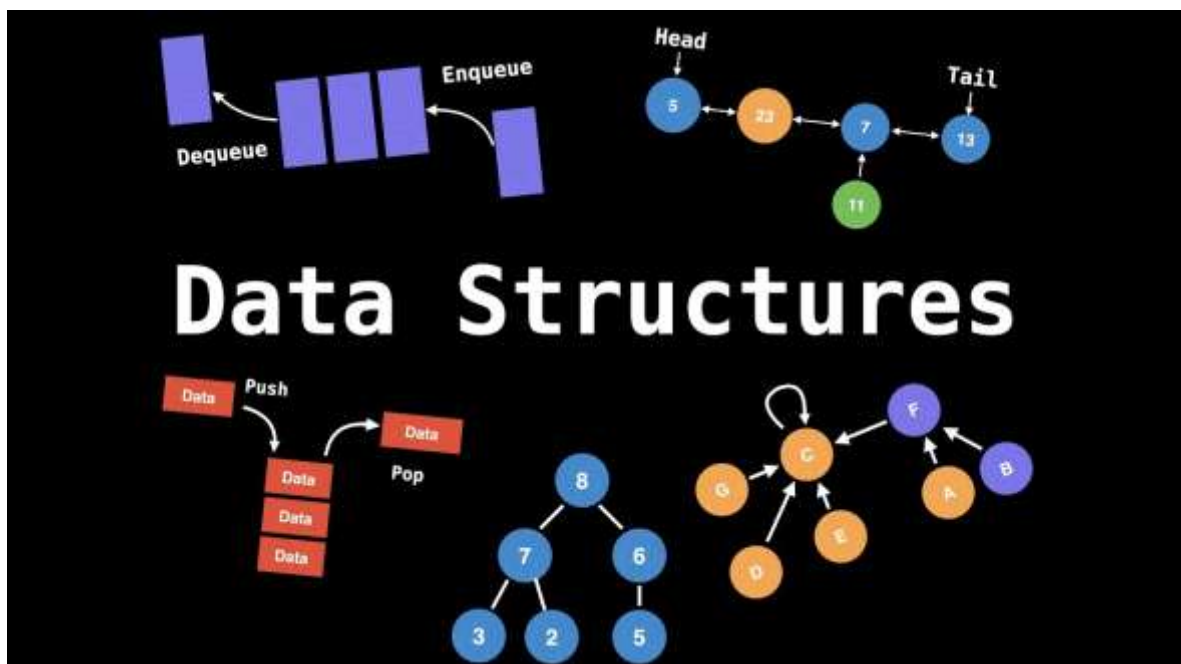
NEW L J INSTITUTE OF ENGINEERING & TECHNOLOGY

SEMESTER- 3

BRANCH- CSE/CSE(AIML)/CSE(DS)

SUBJECT NAME: DATA STRUCTURES

SUBJECT CODE: BE03000081





Course Outcomes

Sr. No.	CO Statement	Chapters	Marks % Weightage
CO-1	Define and classify various data structures, storage structures and common operations on them.	1,4	10
CO-2	Use various linear data structures with their representation and perform different operations on them.	2	20
CO-3	Use various nonlinear data structures with their representation and perform different operations on them.	3	20
CO-4	Apply various searching and sorting techniques on small and large dataset.	5	30
CO-5	Solve the given problem using an appropriate data structure to achieve optimal performance and compare its performance with other possible data structures.	1,2,3,4,5	20

Teaching and Examination Scheme

Teaching - Learning Scheme					Total Credits = TH/30	Assessment Pattern and Marks					Total Marks
L	T	P	TW/SL	TH		Theory		Tutorial / Practical			
						ESE (E)	PA (M)	PA/ (I)	TW/ SL (I)	ESE (V)	
45	0	60	15	120	04	70	30	20	30	50	200

Distribution of Theory Marks

R Level	U Level	A Level	N Level	E Level	C Level

Legends: R: Remembrance; U: Understanding; A: Application, N: Analyse and E: Evaluate C: Create and above Levels (Revised Bloom's Taxonomy)



Syllabus

Sr. No.	Topics	Teaching Hrs.	Module Weightage
1	INTRODUCTION TO DATA STRUCTURES: Basic Terminology, Classification of Data Structure- Primitive and Non Primitive Data Structure, Linear and Non-Linear Data Structures, Examples of Primitive and Non Primitive Data Structures, storage representation of Primitive and Non Primitive Data Structures	04	10
2	LINEAR DATA STRUCTURE Array: Representation of arrays, Applications of arrays, sparse matrix and its representation Stack: Stack-Definitions & Concepts, Operations On Stacks, Applications of Stacks, Polish Expression, Reverse Polish Expression And Their Compilation, Recursion, Tower of Hanoi Queue: Representation Of Queue, Operations On Queue, Circular Queue, Priority Queue, Array representation of Priority Queue, Double Ended Queue, Applications of Queue Linked List: Singly Linked List, Doubly Linked list, Circular linked list, Linked implementation of Stack, Linked implementation of Queue, Applications of linked list.	13	30
3	NON LINEAR DATA STRUCTURE : Trees: Definitions and Concepts, Representation of binary tree, Binary tree traversal (Inorder, postorder, preorder), Threaded binary tree, Binary search trees, Conversion of General Trees To Binary Trees, Applications Of Trees Graphs: Graph-Matrix Representation Of Graphs, Elementary Graph operations,(Breadth First Search, Depth First Search, Spanning Trees, Shortest path, Minimal spanning tree)	13	30
4	HASHING AND FILES TRUCTURES: Hashing: The symbol table, Hashing Functions, Collision- Resolution Techniques File Structure: Fixed and variable length record, File organizations: Sequential, Random Access, indexed and Relative/Random File Organization	07	15
5	Sorting & Searching: Sorting: Various sorting techniques: Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort, Heap Sort Searching: Sequential Search and Binary Search	08	15
	TOTAL	45	100



Unit 5 Sorting & Searching

Syllabus	Sorting: Various sorting techniques: Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort, Heap Sort Searching: Sequential Search and Binary Search
-----------------	---

Basic Terminology

➤ What is sorting?

- **Sorting** is the process of arranging data elements in a specific order, such as ascending or descending. This order can be numerical (e.g., smallest to largest) or alphabetical (e.g., A-Z).
- Sorting is fundamental for efficient data management, making it easier to search, analyze, and process information.
- Examples of Sorting:
 - **Telephone directory:** Names are sorted alphabetically.
 - **Dictionary:** Words are sorted alphabetically.
 - **Database tables:** Rows can be sorted based on specific columns.
 - **E-commerce websites:** Products can be sorted by price, popularity, etc.

➤ Importance of Sorting?

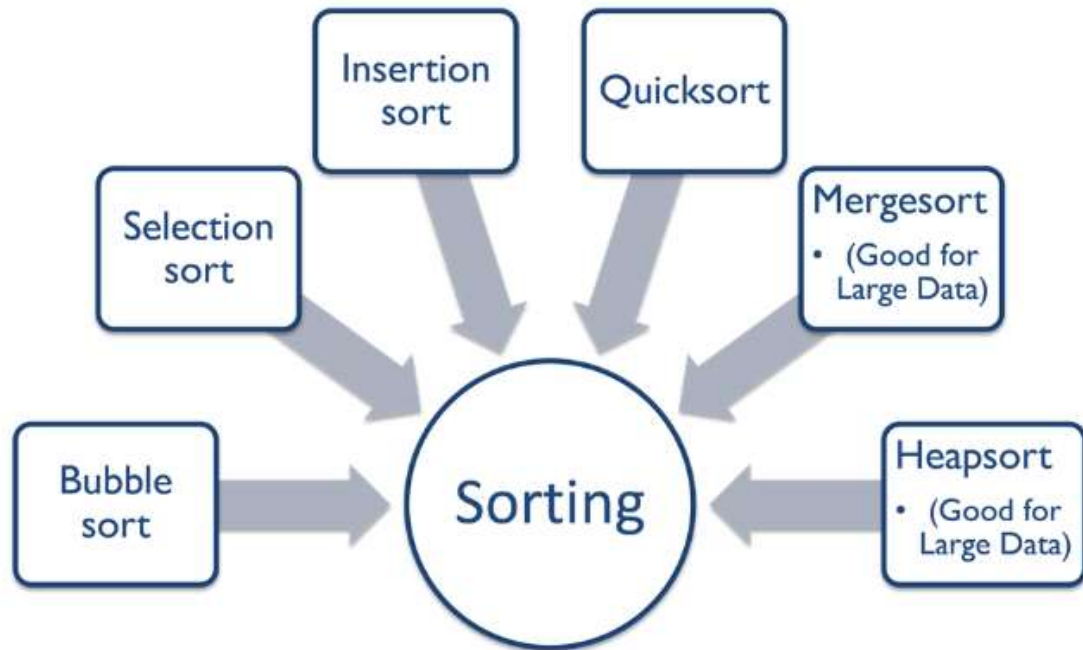
- **Efficient Searching:** Sorted data makes searching for specific elements faster and easier.
- **Data Analysis:** Sorting allows data to better analysis and interpretation.
- **Data Visualization:** Sorting can enhance the visual representation of data.
- **Database Operations:** Sorting is crucial for many database operations, like indexing and query processing.

Types of Sorting Algorithms

➤ Sorting algorithms are often classified by:

- Computational complexity (worst, average and best case) in terms of the size of the list (N). For typical sorting algorithms good behavior is $O(N \log N)$ and worst case behavior is $O(N^2)$ and the average case behavior is $O(N)$.

- Memory Utilization
- Stability - Maintaining relative order of records with equal keys.
- No. of comparisons.
- Methods applied like Insertion, exchange, selection, merging etc.
- There is various sorting algorithms are used in data structures as shown in Figure.



- In data structures, internal sorting refers to sorting algorithms that operate entirely within the computer's main memory (RAM), while external sorting is used when the data to be sorted is too large to fit in memory and requires using external storage like hard drives.
- **Internal Sorting:**
 - **Definition:** Internal sorting algorithms process data that is stored in the main memory.
 - **Applicability:** Suitable for datasets that are relatively small and can be loaded into RAM at once.
 - **Advantages:** Generally faster than external sorting because accessing main memory is quicker than accessing external storage.
 - **Examples:** Bubble Sort, Insertion Sort, Quick Sort, Heap Sort.
- **External Sorting:**
 - **Definition:** External sorting algorithms are designed for datasets that exceed the capacity of main memory.
 - **Applicability:** Used when dealing with large datasets that must be stored on disk or other external storage.

- **Process:** Data is typically read into memory in chunks, processed, and then written back to external storage.
- **Advantages:** Enables sorting of massive datasets that cannot be accommodated in RAM.
- **Examples:** Merge Sort, External Merge Sort.

Bubble sort Algorithm

➤ Bubble sort:

- Bubble sort is a **sorting algorithm** that works by repeatedly stepping through lists that need to be sorted, comparing each pair of adjacent items and swapping them if they are in the wrong order.
- In Bubble Sort, the algorithm will take the 1 element of the array and compare the value with the element next to it in the array.
- If the 1st element is larger than the 2nd then the element will swap the positions.
- If it doesn't satisfy the condition, then the algorithm will compare the 2nd element with 3rd. The above process goes on until the last element.
- **Working of Bubble sort Algorithm:**
 - To understand the working of bubble sort algorithm, let's take an unsorted array.
 - We are taking a short and accurate array Let the elements of array are –

A[9] = {54, 26, 93, 17, 77, 31, 44, 55, 20}

Pass 1:

26	54	93	17	77	31	44	55	20	Exchange
26	54	93	17	77	31	44	55	20	No Exchange
26	54	17	93	77	31	44	55	20	Exchange
26	54	17	77	93	31	44	55	20	Exchange
26	54	17	77	31	93	44	55	20	Exchange
26	54	17	77	31	44	93	55	20	Exchange
26	54	17	77	31	44	55	93	20	Exchange
26	54	17	77	31	44	55	20	93	Exchange
26	54	17	77	31	44	55	20	93	93 in its right position

New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 5: Sorting & Searching

Pass 2:

26	54	17	77	31	44	55	20	93	No Exchange
26	17	54	77	31	44	55	20	93	Exchange
26	17	54	77	31	44	55	20	93	No Exchange
26	17	54	31	77	44	55	20	93	Exchange
26	17	54	31	44	77	55	20	93	Exchange
26	17	54	31	44	55	77	20	93	Exchange
26	17	54	31	44	55	20	77	93	Exchange
26	17	54	31	44	55	20	77	93	77 in its right position

Pass 3:

17	26	54	31	44	55	20	77	93	Exchange
17	26	54	31	44	55	20	77	93	No Exchange
17	26	31	54	44	55	20	77	93	Exchange
17	26	31	44	54	55	20	77	93	Exchange
17	26	31	44	54	55	20	77	93	No Exchange
17	26	31	44	54	20	55	77	93	Exchange
17	26	31	44	54	20	55	77	93	55 in its right position

Pass 4:

17	26	31	44	54	20	55	77	93	No Exchange
17	26	31	44	54	20	55	77	93	No Exchange
17	26	31	44	54	20	55	77	93	No Exchange
17	26	31	44	54	20	55	77	93	No Exchange
17	26	31	44	20	54	55	77	93	Exchange
17	26	31	44	20	54	55	77	93	54 in its right position

Pass 5:

17	26	31	44	20	54	55	77	93	No Exchange
17	26	31	44	20	54	55	77	93	No Exchange
17	26	31	44	20	54	55	77	93	No Exchange
17	26	31	20	44	54	55	77	93	Exchange
17	26	31	20	44	54	55	77	93	44 in its right position

Pass 6:

17	26	31	20	44	54	55	77	93	No Exchange
17	26	31	20	44	54	55	77	93	No Exchange
17	26	20	31	44	54	55	77	93	Exchange
17	26	20	31	44	54	55	77	93	31 in its right position

Pass 7:

17	26	20	31	44	54	55	77	93	No Exchange
17	20	26	31	44	54	55	77	93	Exchange
17	20	26	31	44	54	55	77	93	26 in its right position

- **Algorithm**

BUBBLE_SORT(A, N)

Step 1: Repeat Step 2 For I = 1 to N - 1

Step 2: Repeat For J = 1 to N - I

Step 3: IF A[J] > A[J + 1]

SWAP A[J] and A[J+1]

[END OF INNER LOOP]

[END OF OUTER LOOP]

Step 4: EXIT

- **Program**

```
#include <stdio.h>
int main()
{
    int arr[100], n, i, j, temp;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter %d integers:\n", n);
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    // Bubble Sort Algorithm
    for (i = 0; i < n - 1; i++)
    {
```

```
        for (j = 0; j < n - i - 1; j++)  
        {  
            if (arr[j] > arr[j + 1])  
            {  
                temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
    printf("Sorted array in ascending order:\n");  
    for (i = 0; i < n; i++)  
    {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
    return 0;  
}
```

○ **OUTPUT:**

Enter the number of elements: 5

Enter 5 integers:

11

69

98

5

3

Sorted array in ascending order:

3 5 11 69 98

Selection sort Algorithm

➤ Selection sort:

- In selection sort, the smallest value among the unsorted elements of the array is selected in every pass and inserted to its appropriate position into the array.
- It is also the simplest algorithm.
- In selection sort, the first smallest element is selected from the unsorted array and placed at the first position. After that second smallest element is selected and placed in the second position. The process continues until the array is

entirely sorted.

- The average and worst-case complexity of selection sort is $O(n^2)$, where n is the number of items. Due to this, it is not suitable for large data sets. 2
- Working of Selection sort Algorithm:**

$A[7] = \{-1, 5, 3, 9, 12, 4, 8, 23, 15\}$

-1	5	3	9	12	4	8	23	15	First Min = -1 , swap(a[1],a[1])
-1	5	3	9	12	4	8	23	15	Second Min = 3, swap(a[3],a[2])
-1	3	5	9	12	4	8	23	15	Third Min = 4, swap(a[6],a[3])
-1	3	4	9	12	5	8	23	15	Fourth Min = 5, swap(a[6],a[4])
-1	3	4	5	12	9	8	23	15	Fifth Min = 8, swap(a[7],a[5])
-1	3	4	5	8	9	12	23	15	Sixth Min = 9, swap(a[6],a[6])
-1	3	4	5	8	9	12	23	15	Seventh Min = 12, swap(a[7],a[7])
-1	3	4	5	8	9	12	23	15	Eighth Min = 15, swap(a[9],a[8])
-1	3	4	5	8	9	12	15	23	Sorted Array

- Algorithm:**

```

SELECTION_SORT (A)
for i ← 1 to n-1 do
    min ← i;
    for j ← i + 1 to n do
        If A[j] < A[i] then
            min ← j
    If min != i then
        temp ← A[i]
        A[i] ← A [min]
        A[min] ← temp

```

- Program:**

```

#include <stdio.h>
int main()
{
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
    {

```

```
        scanf("%d", &arr[i]);
    }
    // Selection Sort algorithm
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }
        if (minIndex != i)
        {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

- **OUTPUT:** Enter the number of elements: 7

Enter 7 elements:

98

12

66

58

43

9

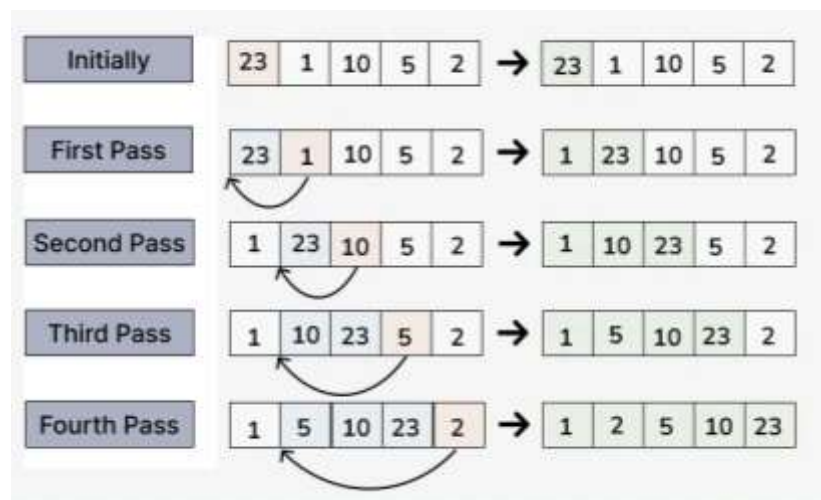
28

Sorted array: 9 12 28 43 58 66 98

Insertion sort Algorithm

➤ Insertion sort:

- In insertion sort, every iteration moves an element from unsorted portion to sorted portion until all the elements are sorted in the list.
- Assume that first element in the list is in sorted portion of the list and remaining all elements are in unsorted portion.
- Select first element from the unsorted list and insert that element into the sorted list in order specified.
- Repeat the above process until all the elements from the unsorted list are moved into the sorted list.
- This algorithm is not suitable for large data sets.
- **Working of Insertion sort Algorithm**



• Algorithm:

INSERTION-SORT (ARR, N)

Step 1: Repeat Steps 2 to 5 for K = 1 to N -1

Step 2: SET TEMP =ARR[K]

Step 3: SET J = K - 1

Step 4: Repeat while TEMP <= ARR[J]

SET ARR[J +1] = ARR[J]

SET J = J - 1

[END OF INNER LOOP]

Step 5: SET ARR[J + 1] = TEMP
[END OF LOOP]

Step 6: EXIT

- **Program:**

```
#include <stdio.h>
int main()
{
    int n, k, j, temp, arr[n];
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter %d integers:\n", n);
    for (k = 0; k < n; k++)
    {
        scanf("%d", &arr[k]);
    }
    for (k = 1; k < n; k++)
    {
        temp = arr[k];
        j = k - 1;
        while (j >= 0 && temp <= arr[j])
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = temp;
    }
    printf("Sorted array in ascending order:\n");
    for (k = 0; k < n; k++)
    {
        printf("%d ", arr[k]);
    }
    printf("\n");
    return 0;
}
```

- **OUTPUT:** Enter the number of elements: 6
Enter 6 integers:

79

82

17

25

60

53

Sorted array in ascending order:

17 25 53 60 79 82

Quick Sort Algorithm

➤ Quick Sort:

- Quicksort is the widely used sorting algorithm. It is a faster and highly efficient sorting algorithm. This algorithm follows the divide and conquer approach. **Divide and conquer** is a **technique** of breaking down the algorithms into subproblems, then solving the subproblems, and combining the results back together to solve the original problem.
- **Quicksort picks an element as pivot**, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot.
- After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.
- **Working of Quicksort Algorithm:**



Step : 1

	0	1	2	3	4	5	6	7
A	50	30	10	90	80	20	40	70

Pivot

$A[i] \leq \text{pivot} (50 \leq 50)$

$i++$

Step : 2

	0	1	2	3	4	5	6	7
A	50	30	10	90	80	20	40	70

Pivot

$A[i] \leq \text{pivot} (30 \leq 50)$

$i++$

Step : 3

	0	1	2	3	4	5	6	7
A	50	30	10	90	80	20	40	70

Pivot

$A[i] \leq \text{pivot} (10 \leq 50)$

$i++$

Step : 4

	0	1	2	3	4	5	6	7
A	50	30	10	90	80	20	40	70

Pivot

$A[i] \leq \text{pivot} (90 \leq 50) \rightarrow \text{NO}$

$A[j] \geq \text{pivot} (70 \geq 50)$

$i - \text{stop incremented}$

$j--$

Step : 5

	0	1	2	3	4	5	6	7
A	50	30	10	90	80	20	40	70

Pivot

$A[j] \geq \text{pivot} (40 \geq 50) \rightarrow \text{No}$

$i - \text{decrement Stopped}$
 $j - \text{decrement stopped}$

$j - \text{decrement Stopped}$

Swap (90 , 40)

Step : 6

	0	1	2	3	4	5	6	7
A	50	30	10	40	80	20	90	70

Pivot

$A[i] \leq \text{pivot} (40 \leq 50)$

$i++$

Step : 7

	0	1	2	3	4	5	6	7
A	50	30	10	40	80	20	90	70

Pivot

$A[i] \leq \text{pivot} (80 \leq 50) \rightarrow \text{No}$

$A[j] \geq \text{pivot} (90 \geq 50)$

$i - \text{increment Stopped}$

$j--$

Step : 8

	0	1	2	3	4	5	6	7
A	50	30	10	40	80	20	90	70

Pivot

$A[j] \geq \text{pivot} (20 \geq 50) \rightarrow \text{No}$

$i - \text{decrement Stopped}$
 $j - \text{decrement stopped}$

$j - \text{decrement Stopped}$

Swap (20 , 80)

Step : 9

	0	1	2	3	4	5	6	7
A	50	30	10	40	20	80	90	70

Pivot

$A[i] \leq \text{pivot} (20 \leq 50)$

$i++$

Step : 10

	0	1	2	3	4	5	6	7
A	50	30	10	40	20	80	90	70

Pivot

$A[i] \leq \text{pivot} (80 \leq 50) \rightarrow \text{No}$

$A[j] \geq \text{pivot} (80 \geq 50)$

$i - \text{decrement Stopped}$

$j--$

Step : 11

	0	1	2	3	4	5	6	7
A	50	30	10	40	20	80	90	70

Pivot

$j \text{ crosses } - i$

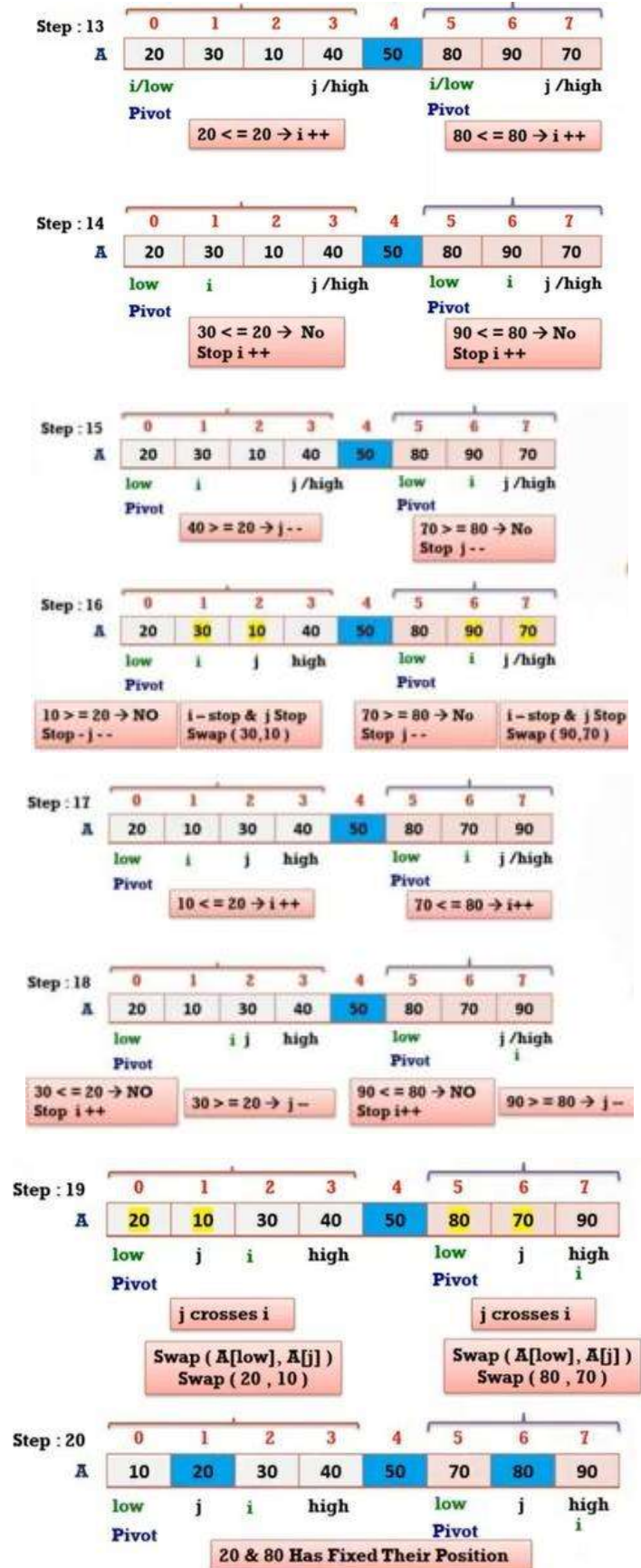
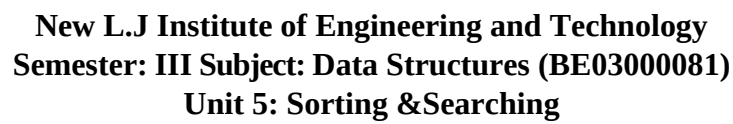
Swap(A[j], A[low] = Swap (20 , 50)

Step : 12

	0	1	2	3	4	5	6	7
A	20	30	10	40	50	80	90	70

Pivot

50 - Fixed
Divide From 50



- **Algorithm:**

Quicksort Algorithm

```
Algorithm QuickSort(A[0...n-1], low, high)
{
    if (low < high)
    {
        m ← partition(A[low...high])
        QuickSort(A[low...m-1])
        QuickSort(A[m+1...high])
    }
}
```

Partition Algorithm

```
Algorithm Partition(A[low...high])
{
    pivot ← A[low]
    i ← low
    j ← high
    while (i <= j)
    {
        while (A[i] <= pivot)
        {
            i++
        }
        while (A[j] >= pivot)
        {
            j--
        }
        if (i <= j) { swap(A[i], A[j]) }
    }

    swap(A[low], A[j]) // when i crosses j
    return j
}
```

- **Program:**

```
#include <stdio.h>
// Function to swap two numbers
```

```
void swap(int arr[], int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}

// Partition function (pivot = first element)
int partition(int arr[], int low, int high) {
    int pivot = arr[low];
    int i = low + 1;
    int j = high;
    while (1) {
        while (i <= high && arr[i] <= pivot) { i++; }
        while (arr[j] > pivot) { j--; }
        if (i < j) {
            swap(arr, i, j);
        } else {
            break;
        }
    }
    swap(arr, low, j);
    return j;
}

// QuickSort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high); // partition index
        quickSort(arr, low, pi - 1);      // sort left side
        quickSort(arr, pi + 1, high);      // sort right side
    }
}

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements: ", n);
```

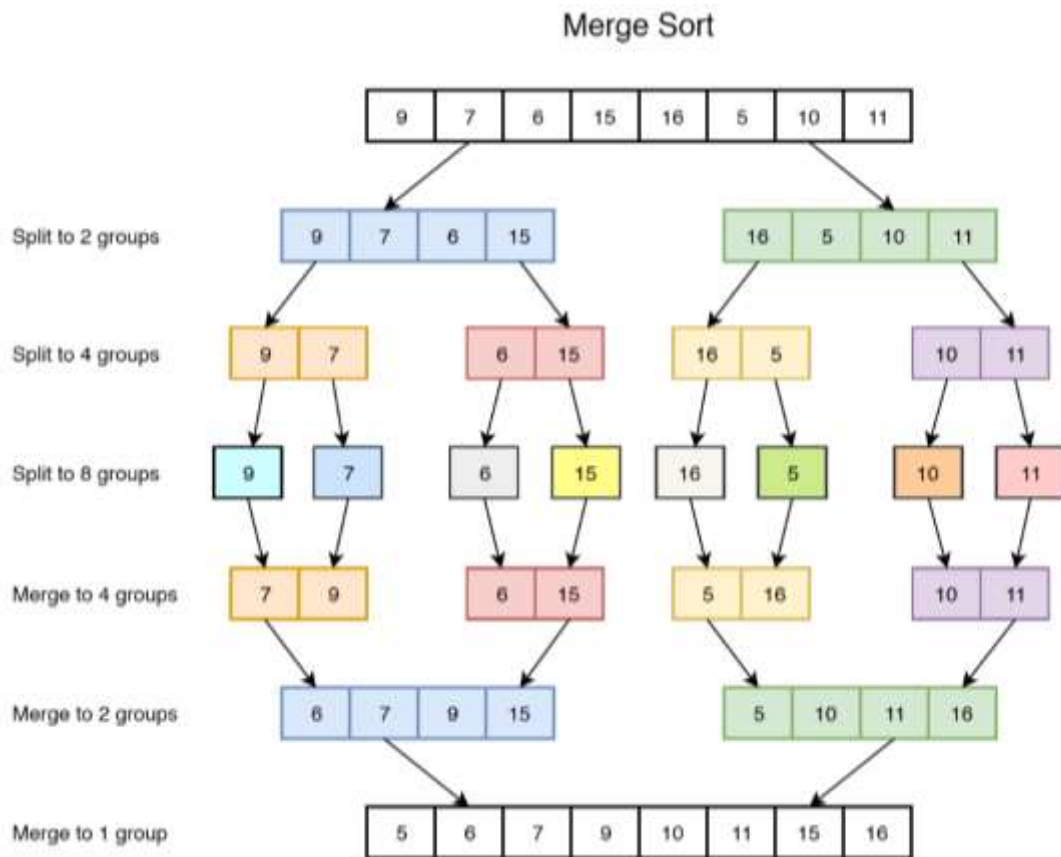
```
for (i = 0; i < n; i++)  
    scanf("%d", &arr[i]);  
quickSort(arr, 0, n - 1);  
printf("Sorted array: ");  
for (i = 0; i < n; i++)  
    printf("%d ", arr[i]);  
printf("\n");  
return 0;  
}  
  
○ OUTPUT : Enter number of elements: 5  
Enter 5 elements:  
32  
5  
89  
4  
1  
Sorted Array: 1 4 5 32 89
```

Merge Sort Algorithm

➤ Merge sort:

- Merge sort is a sorting algorithm that employs the "**divide and conquer**" strategy.
- It recursively divides an array into smaller sub-arrays, sorts them, and then merges the sorted sub-arrays to produce a final sorted array.
- **Divide:** The input array is repeatedly divided into two halves until each sub-array contains only one element (which is considered sorted).
- **Conquer:** The algorithm sorts these single-element sub-arrays (implicitly, as they are already sorted).
- **Combine (Merge):** The sorted sub-arrays are merged in a sorted manner to produce larger sorted sub-arrays, eventually merging them all into a single, fully sorted array.

- Working of Mergesort Algorithm:



- Algorithm:**

Algorithm MergeSort(A[0...n-1], low, high)

```
{
    if (low < high)
    {
        mid ← (low + high / 2 )
        MergeSort(A, low, mid)
        MergeSort(A, mid + 1, high)
        Combine(A, low, mid, high)
    }
}
```

Algorithm Combine(A[0...n-1], low, mid, high)

```
{
    k ← low // index for array setup
    i ← low
    j ← mid + 1
    while (i <= mid and j <= high)
```

```
{  
    if (A[i] <= A[j])  
    {  
        temp[k] = A[i]  
        i++  
        k++  
    }  
    else  
    {  
        temp[k] = A[j]  
        j++  
        k++  
    }  
}  
while (i <= mid)  
{  
    temp[k] = A[i]  
    i++  
    k++  
}  
while (j <= high)  
{  
    temp[k] = A[j]  
    j++  
    k++  
}  
}
```

- **Program:**

```
#include <stdio.h>  
  
// Merge two halves  
void merge(int arr[], int left, int mid, int right) {  
    int i = left, j = mid + 1, k = 0;  
    int temp[right - left + 1];  
  
    while (i <= mid && j <= right) {
```

```
        if (arr[i] <= arr[j]) {
            temp[k] = arr[i];
            i++;
            k++;
        } else {
            temp[k] = arr[j];
            j++;
            k++;
        }
    }

    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= right) {
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (i = left, k = 0; i <= right; i++, k++) {
        arr[i] = temp[k];
    }
}

// Merge Sort function
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);    // sort left half
        mergeSort(arr, mid + 1, right); // sort right half

        merge(arr, left, mid, right); // merge sorted halves
    }
}
```

```
}  
}  
  
int main() {  
    int n;  
    printf("Enter number of elements: ");  
    scanf("%d", &n);  
  
    int arr[n];  
    printf("Enter %d elements:\n", n);  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }  
  
    mergeSort(arr, 0, n - 1);  
  
    printf("\nSorted Array:\n");  
    for (int i = 0; i < n; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
  
    return 0;  
}
```

- **OUTPUT:** Enter number of elements: 5

Enter 5 elements:

12

1

9

5

4

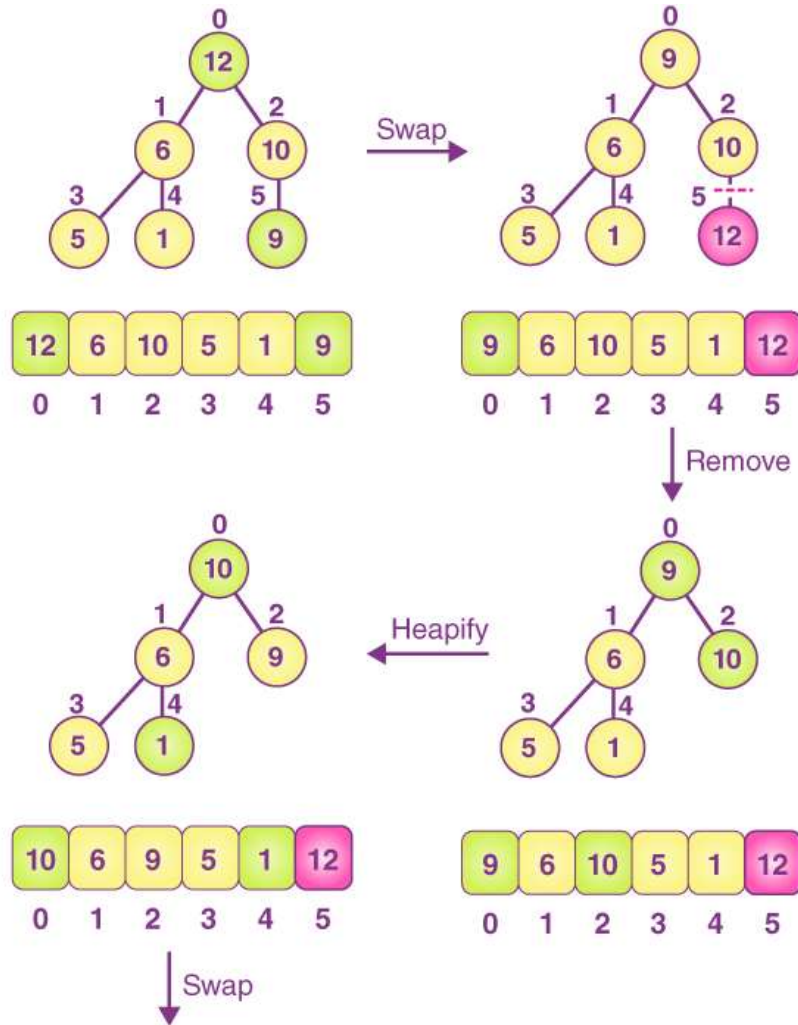
Sorted Array:

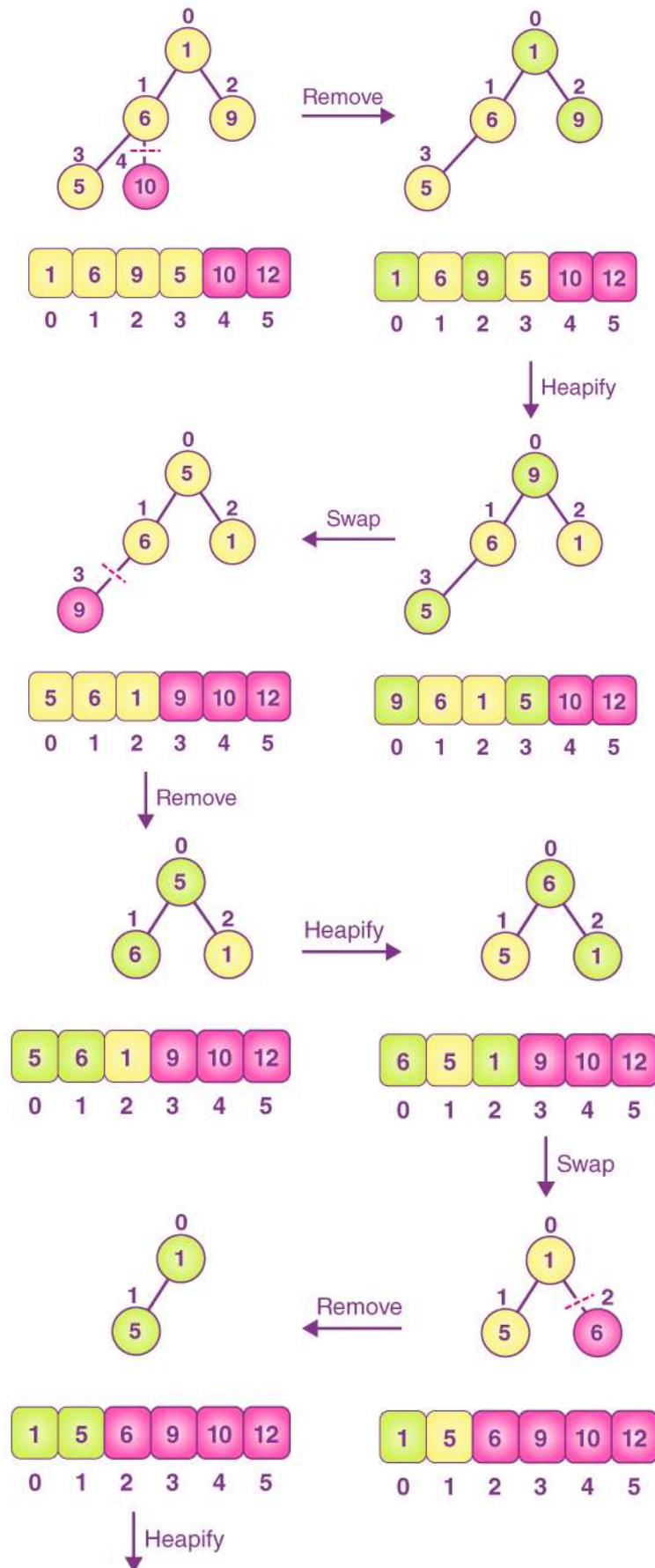
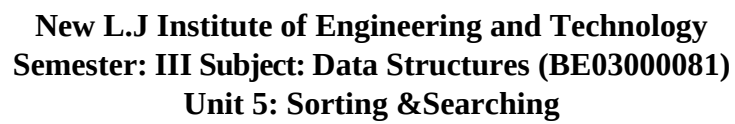
1 4 5 9 12

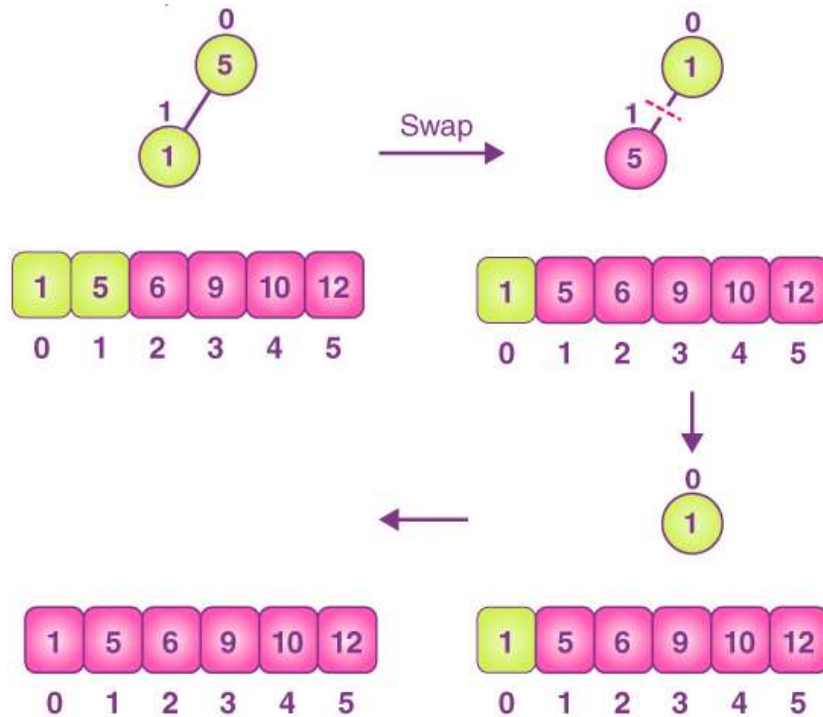
Heap Sort Algorithm

➤ Heap Sort:

- Heap sort is a way to sort a list of items, like numbers, in order.
- It uses a special tree structure called a heap.
- A heap is a kind of binary tree where each parent node is greater than or equal to its child nodes. This helps in easily finding the largest or smallest item.
- Heaps are usually represented as binary heaps and come in two main types:
 1. **Max heap:** In a max heap, the value of the parent node is greater than or equal to the values of its children. This ensures that the largest element is always at the root.
 2. **Min heap:** Nodes that have a value less than or equal to the value of their children are called min heaps. The smallest node in a min-heap is at the root.
- **Working of Heapsort Algorithm:**
 - **Build a Heap:** First, we arrange the list of numbers into a heap. This makes sure the largest number is at the top of the heap.
 - **Remove the Top:** Then, we remove the top (the largest number) and place it at the end of the list.
 - **Rebuild the Heap:** After removing the top, we rebuild the heap with the remaining numbers.
 - **Repeat:** We keep repeating the process of removing the top and rebuilding the heap until all numbers are sorted.
 - **Example:** Apply Heapsort algorithm to the following elements 12 , 6 , 10 , 5, 1 ,9
 - Building a heap using the BUILD-MAX-HEAP algorithm from the input array –







- **Algorithm:**

HEAPSORT(ARR, N)

Step 1: [Build Heap H]

Repeat for I = to N-1

CALL Insert_Heap(ARR, N, ARR[I])

[END OF LOOP]

Step 2: (Repeatedly delete the root element)

Repeat while N > 0

CALL Delete_Heap(ARR, N, VAL)

SET N= N + 1

[END OF LOOP]

Step 3: END

- **Program:**

```
#include <stdio.h>
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
void heapify(int arr[], int n, int i)
{

```

```
int largest = i;
int left = 2 * i + 1;
int right = 2 * i + 2;
if (left < n && arr[left] > arr[largest])
{
    largest = left;
}
if (right < n && arr[right] > arr[largest])
{
    largest = right;
}
if (largest != i)
{
    swap(&arr[i], &arr[largest]);
    heapify(arr, n, largest);
}
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
    {
        heapify(arr, n, i);
    }
    for (int i = n - 1; i > 0; i--)
    {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
int main()
{
    int numbers[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(numbers) / sizeof(numbers[0]);
    printf("Original array: ");
    printArray(numbers, n);
    heapSort(numbers, n);
    printf("Sorted array: ");
    printArray(numbers, n);
    return 0;
}
```

- **OUTPUT:** Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13

Complexity

➤ Time Complexity:

- **Time complexity** measures the amount of time an algorithm takes to complete as a function of the input size. It's a way to estimate the running time of an algorithm.
- **Big O Notation (O-notation):** This is used to describe the upper bound of an algorithm's running time. It tells you how the runtime scales with the size of the input.
 - **O(1):** Constant time. The running time doesn't change with the input size. Example: Accessing an array element by index.
 - **O(log n):** Logarithmic time. The running time increases logarithmically as the input size increases. Example: Binary search in a sorted array.
 - **O(n):** Linear time. The running time increases linearly with the input size. Example: Iterating through an array.
 - **O(n log n):** Log-linear time. Common in efficient sorting algorithms like merge sort and quicksort.
 - **O(n²):** Quadratic time. The running time grows quadratically with the input size. Example: Nested loops, such as in bubble sort.
 - **O(2ⁿ):** Exponential time. The running time doubles with each additional input element. Example: Recursive algorithms solving the Towers of Hanoi.

- **$O(n!)$** : Factorial time. The running time grows factorially with the input size. Example: Brute-force solutions to the traveling salesman problem.
- **Key Points:**
 - **Best Case**: The scenario where the algorithm performs the minimum number of steps.
 - **Worst Case**: The scenario where the algorithm performs the maximum number of steps.
 - **Average Case**: The expected scenario, considering all possible inputs.
- **Space Complexity:**
 - **Space complexity** measures the amount of memory an algorithm uses as a function of the input size.
 - **Big O Notation for Space Complexity**: Just like time complexity, space complexity can be expressed in Big O notation.
 - **$O(1)$** : Constant space. The algorithm uses a fixed amount of memory regardless of input size. Example: Variables storing sums or counters.
 - **$O(n)$** : Linear space. The memory usage grows linearly with the input size. Example: Storing an additional array or list.
 - **$O(n^2)$** : Quadratic space. The memory usage grows quadratically with the input size. Example: Storing a 2D matrix.
 - **Key Points:**
 - **Auxiliary Space**: The extra space or temporary space used by an algorithm.
 - **In-place Algorithm**: An algorithm that uses a constant amount of extra space, typically $O(1)$

Sorting Algorithms	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

Linear /Sequential Search

➤ Linear/Sequential Search

- linear search or sequential search is a method for finding a particular value in a list that consists of checking every one of its elements, one at a time and in sequence, until the desired one is found.
- Linear search is the simplest search algorithm. It is a special case of brute-force search.
- Its worst case cost is proportional to the number of elements in the list.
- **Best Case Complexity** - In Linear search, best case occurs when the element we are finding is at the first position of the array. The best-case time complexity of linear search is **$O(1)$** .
- **Average Case Complexity** - The average case time complexity of linear search is **$O(n)$** .
- **Worst Case Complexity** - In Linear search, the worst case occurs when the element we are looking is present at the end of the array. The worst-case in linear search could be when the target element is not present in the given array, and we have to traverse the entire array. The worst-case time complexity of linear search is **$O(n)$** .
- The time complexity of linear search is **$O(n)$** because every element in the array is compared only once.
- The space complexity of linear search is $O(1)$.
- **Working on Linear Search:** Let's understand the working of linear search with an example of an unsorted array:

- Here we have an array of elements

0	1	2	3	4	5	6	7	8
29	87	89	21	23	17	11	10	14

- Let **$K = 17$** be the element to be searched. Now, we will compare K with each of the elements until we find $K=17$

0	1	2	3	4	5	6	7	8
29	87	89	21	23	17	11	10	14

↑
 $K \neq 17$

- The value of K is not matched with the first element of the array. So, we will switch to the next element and keep repeating the same step until $K=17$.

0	1	2	3	4	5	6	7	8
29	87	89	21	23	17	11	10	14

↑
 $K \neq 87$

0	1	2	3	4	5	6	7	8
29	87	89	21	23	17	11	10	14

↑
 $K \neq 89$

0	1	2	3	4	5	6	7	8
29	87	89	21	23	17	11	10	14

↑
 $K \neq 21$

0	1	2	3	4	5	6	7	8
29	87	89	21	23	17	11	10	14

↑
 $K \neq 23$

0	1	2	3	4	5	6	7	8
29	87	89	21	23	17	11	10	14

↑
 $K = 17$

- Now, as the element to be searched, i.e., $K=17$ is found, it will return the index of the matched element.

- **Algorithm:**

LINEAR_SEARCH(A, N, VAL)

Step 1: [INITIALIZE] SET POS = -1

Step 2: [INITIALIZE] SET I = 1

Step 3: Repeat Step 4 while I ≤ N

Step 4: IF A[I] = VAL
SET POS = I
PRINT POS
Go to Step 6

[END OF IF]

SET I = I + 1

[END OF LOOP]

Step 5: IF POS = -1

PRINT "value is not present in the array"

[END OF IF]

Step 6: EXIT

- **Program:**

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int array[100], key, i, n;
```

```
    printf("Enter the number of elements in array\n");
```

```
    scanf("%d",&n);
```

```
    printf("Enter %d integer(s)\n", n);
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        printf("Array[%d]=", i);
```

```
        scanf("%d", &array[i]);
```

```
    }
```

```
    printf("Enter the number to search\n");
```

```
    scanf("%d", &key);
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        if (array[i] == key)
```

```
        /* if required element found */
```

```
        {
```

```
            printf("%d is present at location %d.\n", key, i+1);
```

```
        break;
    }
}
if (i == n)
{
    printf("%d is not present in array.\n", key);
}
getch();
}
```

- **OUTPUT:** Enter the number of elements in array
5
Enter 5 integer(s)
Array[0]=12
Array[1]=54
Array[2]=74
Array[3]=69
Array[4]=32
Enter the number to search
69
69 is present at location 4.

Binary Search

➤ Binary Search :

- The Binary Search algorithm is a fast technique that works efficiently on a sorted list.
- It is important to make sure that the list should be a sorted one from which the element is to be searched.
- Binary search **works on the divide and conquer** approach.
- The list from which the search is to be done is divided into two halves, and then the searched element is compared with the middle element in the array. If the element is found, then the index of the middle element is returned.
- Otherwise, the search will keep going in either of the halves according to the result generated through the match.
- **Best Case Complexity** - In Binary search, best case occurs when the element to search is found in first comparison, i.e., when the first middle

element itself is the element to be searched. The best-case time complexity of Binary search is **O(1)**.

- **Average Case Complexity** - The average case time complexity of Binary search is **O(logn)**.
- **Worst Case Complexity** - In Binary search, the worst case occurs, when we have to keep reducing the search space till it has only one element. The worst-case time complexity of Binary search is **O(logn)**.
- The space complexity of binary search is O(1).
- **Working of Binary Search Algorithm:**

○ Now, let's understand the working of the Binary Search Algorithm.

Let's take an example of a sorted array:

○ Let the elements of the array are –

0	1	2	3	4	5	6	7	8
11	13	25	28	35	45	55	60	65

○ Let the element to search is, **K = 60**

○ Now, we need to find out mid of the array with the formula:

○ $(\text{Start} + \text{finish}) / 2$

○ So, in the given array **start** = 0 and **finish** = 8

○ **mid** = $(0 + 8) / 2 = 4$. Thus, 4 is in the middle of the array.

0	1	2	3	4	5	6	7	8
11	13	25	28	35	45	55	60	65

↑
A[mid]=35
A[mid]<K(or, 35<60)
So, Start = mid+1, Finish=8
Now, mid = (beg+end)/2 = 13/2 = 6

0	1	2	3	4	5	6	7	8
11	13	25	28	35	45	55	60	65

↑
A[mid]=55
A[mid]<K(or, 55<60)
So, Start = mid+1 = 7, Finish=8
Now, mid = (beg+end)/2 = 15/2 = 7

0	1	2	3	4	5	6	7	8
11	13	25	28	35	45	55	60	65

↑
A[mid]=60
A[mid]=K(or,60=60)
So, position = mid
Now, we got the element
at the 7th position of the
array

- As we have found here the searched element, the index of the matched element will return

- **Algorithm:**

BINARY_SEARCH(A, lower_bound, upper_bound, VAL)

Step 1: [INITIALIZE] SET BEG =lower_bound

END = upper_bound, POS = - 1

Step 2: Repeat Steps 3 and 4 while BEG <= END

Step 3: SET MID = (BEG +END)/2

Step 4: IF A[MID] =VAL
SET POS =MID
PRINT POS
Go to Step 6
ELSE IF A[MID] >VAL
SET END =MID -1
ELSE
SET BEG =MID + 1
[END OF IF]
[END OF LOOP]

Step 5: IF POS = - 1
PRINT "VALUE IS NOT PRESENT IN THE ARRAY"
[END OF IF]

Step 6: EXIT

- **Program:**

```
#include <stdio.h>
void main()
```

```
{
int i, first, last, middle, n, key,
array[100];
printf("Enter number of elements\n");
scanf("%d",&n);
printf("Enter %d integers in sorted order\n", n);
for ( i = 0 ; i < n ; i++ )
{
    scanf("%d",&array[i]);
}
printf("Enter value to find\n");
scanf("%d",&key);
first = 0;
last = n - 1;
middle = (first+last)/2;
while( first <= last )
{
    if (array[middle] == key)
    {
        printf("%d found at location %d.\n", key, middle+1);
        break;
    }
    else if ( array[middle]>key )
    {
        last=middle - 1;
        middle=(first+last)/2;
    }
    else
    {
        first = middle + 1;
        middle=(first+last)/2;
    }
}
if ( first > last )
{
    printf(" %d is not present in the list.\n", key);
}
```

```
getch();
```

```
}
```

○ **OUTPUT:** Enter number of elements

3

Enter 3 integers in sorted order

45

12

32

Enter value to find

12

12 found at location 2.

Difference Between Linear Search and Binary Search

Linear Search	Binary Search
In linear search input data need not to be in sorted.	In binary search input data need to be in sorted order.
It is also called sequential search.	It is also called half-interval search.
The time complexity of linear search O(n) .	The time complexity of binary search O(log n) .
Multidimensional array can be used.	Only single dimensional array is used.
Linear search performs equality comparisons.	Binary search performs ordering comparisons.
It is less complex.	It is more complex.
It is very slow process.	It is very fast process.

GTU Questions from Unit 5

Sr. No	Questions	CO	Answer	Marks
1	What do you mean by internal and external sorting? [NLJIET]	4,5	Pg 2,3	3
2	Write algorithm for Bubble sort method. [NLJIET]	4,5	Pg 5	3
3	Write c program for bubble sort. [NLJIET]	4,5	Pg 5,6	3
4	Write an algorithm for selection sort. [NLJIET]	4,5	Pg 7	3
5	Write c program for selection sort.[NLJIET]	4,5	Pg 7,8,9	3
6	Compare linear search and binary search in terms of their time complexity. [NLJIET]	4,5	Pg 35	3
7	Search the number 50 from the given data using binary search technique. Illustrate the searching process.10, 14, 20, 39, 41, 45, 49, 50, 60 [NLJIET]	4,5	Pg 31,32,33	3
8	Explain binary search technique. [NLJIET]	4,5	Pg 31,32,33	3
9	Does a pivot selection method affect the time complexity of quick sort? Justify your answer. [NLJIET] Answer:Yes, the pivot selection method significantly affects the time complexity of Quick Sort. Justification: • Worst Case: $O(n^2)$ ○ If the pivot is consistently chosen as the smallest or largest element in the sub-array, Quick Sort degrades to its worst-case time complexity of $O(n^2)$. ○ In this scenario, one of the partitions will always be empty (or contain only one element), and the other partition will contain $n-1$ elements. This effectively turns Quick Sort into a Selection Sort or	4,5	-	3

	Insertion Sort-like process, where each element is compared with almost all other elements. • Example: If the array is already sorted and the last element is always chosen as the pivot, or if the array is sorted in descending order and the first element is always chosen as the pivot. • Best and Average Case: $O(n \log n)$ ○ Quick Sort achieves its best and average-case time complexity of $O(n \log n)$ when the pivot selection method leads to balanced partitions. ○ Ideally, a good pivot divides the array into two sub-arrays of roughly equal size. When this happens, the recursion depth is logarithmic ($\log n$), and at each level of recursion, approximately 'n' comparisons are made in total. ○ Methods like "median-of-three" pivot selection, or choosing a random pivot, aim to achieve this balance and avoid the worst-case scenario with a high probability.			
10	Write an algorithm for Bubble sort(Dec-2025)[NLJIET]	4,5	Pg 3,4,5	3
11	Sort following elements using quick sort. 22 15 26 06 40 31, 92, 45, 78.(Dec-2025)[NLJIET]	4,5	Pg 11,12, 13,14	3
12	Explain bubble sort with suitable example. [NLJIET]	4,5	Pg 3,4,5	4
13	Write a C program for a selection sort. [NLJIET]	4,5	Pg 7,8,9	4
14	Sort the elements 77, 49, 25, 12, 9, 33, 56, 81 using selection sort..(June-2026)[NLJIET]	4,5	Pg 7,8,9	4
15	Sort the following array elements using insertion sort algorithm. 8, -2, 5, 3, 9, 4, 6	4,5	Pg 9,10	4



New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 5: Sorting & Searching

	[NLJIET]			
16	Sort 20,35,40,100,3,10,15 using insertion sort. Show all passes. [NLJIET]	4,5	Pg 9,10	4
17	Write an algorithm for insertion sort. [NLJIET]	4,5	Pg 10	4
18	Explain fastest sorting technique with example. [NLJIET]	4,5	Pg 11,12, 13,14,15	4
19	Sort the given values using Quick Sort? 65, 70, 75, 80, 85, 60, 55, 50, 45. Show all passes. [NLJIET]	4,5	Pg 11,12, 13,14	4
20	Sort the following data using quick sort. 50, 30,80,40, 35,70, 60, 20,75 [NLJIET]	4,5	Pg 11,12, 13,14	4
21	Sort following data using quick sort. 10 15 28 09 39 31 30 14 07 08 [NLJIET]	4,5	Pg 11,12, 13,14	4
22	Write an algorithm for quick sort. [NLJIET]	4,5	Pg 14,15	4
23	Apply merge sort algorithm to the following elements. 20, 10, 5, 15, 25, 30,50, 35. [NLJIET]	4,5	Pg 17	4
24	Sort the following data using merge sort.50, 20, 70, 05, 30, 80, 55, 25 [NLJIET]	4,5	Pg 17	4
25	Sort following data using merge sort. 10 15 28 09 39 31 30 14 07 08[NLJIET]	4,5	Pg 17	4
26	Write algorithm for Merge sort method.[NLJIET]	4,5	Pg 18,19	4
27	Apply merge sort on the following array. A=[38, 27, 43, 10].(June-2026)[NLJIET]	4,5	Pg 17	4
28	Define time and space complexity. Derive time complexity of merge sort. [NLJIET]	4,5	Pg 27	4
29	Explain Binary Search with example. [NLJIET]	4,5	Pg 32,33	4
30	Explain the Binary search method.(Dec-2025)[NLJIET]	4,5	Pg 32,33	4
31	What is the worst case complexity of binary search? Write an algorithm for binary search. [NLJIET]	4,5	Pg 33	4
32	Compare sequential searching with binary searching in detail. [NLJIET]	4,5	Pg 35	4
33	Write a 'C' program for bubble sort. [NLJIET]	4,5	Pg 5,6	7



New L.J Institute of Engineering and Technology
Semester: III Subject: Data Structures (BE03000081)
Unit 5: Sorting & Searching

34	Explain the bubble sort algorithm. How can you optimize bubble sort to reduce the number of swaps?(June-2026)[NLJIET]	4,5	Pg 3,4,5,6	7
35	Examine the algorithm for Insertion sort and bubble sort the following array: 77, 33, 44, 11, 88, 22, 66, 55. [NLJIET]	4,5	Pg 3,4, 5,9	7
36	Apply quick sort for the following data: 9, 7, 5, 11, 12, 2, 14, 3, 10, 6 [NLJIET]	4,5	Pg 12, 13,14	7
37	Do as directed below: i) Sort following elements using quick sort. 10 15 28 09 40 35 29 12 06 07 ii) State time and space complexity of quick sort. [NLJIET]	4,5	Pg 12, 13,14	7
38	What is time and space analysis? State and explain time analysis for linear search and binary search method. [NLJIET]	4,5	Pg 28,32	7
39	Write an algorithm for merge sort.(Jun-2026)	4,5	Pg 18,19	7
40	Write an algorithm to sort an array using selection sort method.(Dec-2025)[NLJIET]	4,5	Pg 7,8,9	7
41	Sort the following numbers using (i) Selection sort, (ii)Quick sort: [10, 50, 0, 20, 30, 10]. Show the steps.(Dec-2025)[NLJIET]	4,5	Pg 7,8,9,12,13,14	7